Information Access with Apache Lucene - Part 1

Metodi per il Ritrovamento dell'Informazione

Laurea Triennale in Informatica

Università degli Studi di Bari Aldo Moro

Prof. Cataldo Musto

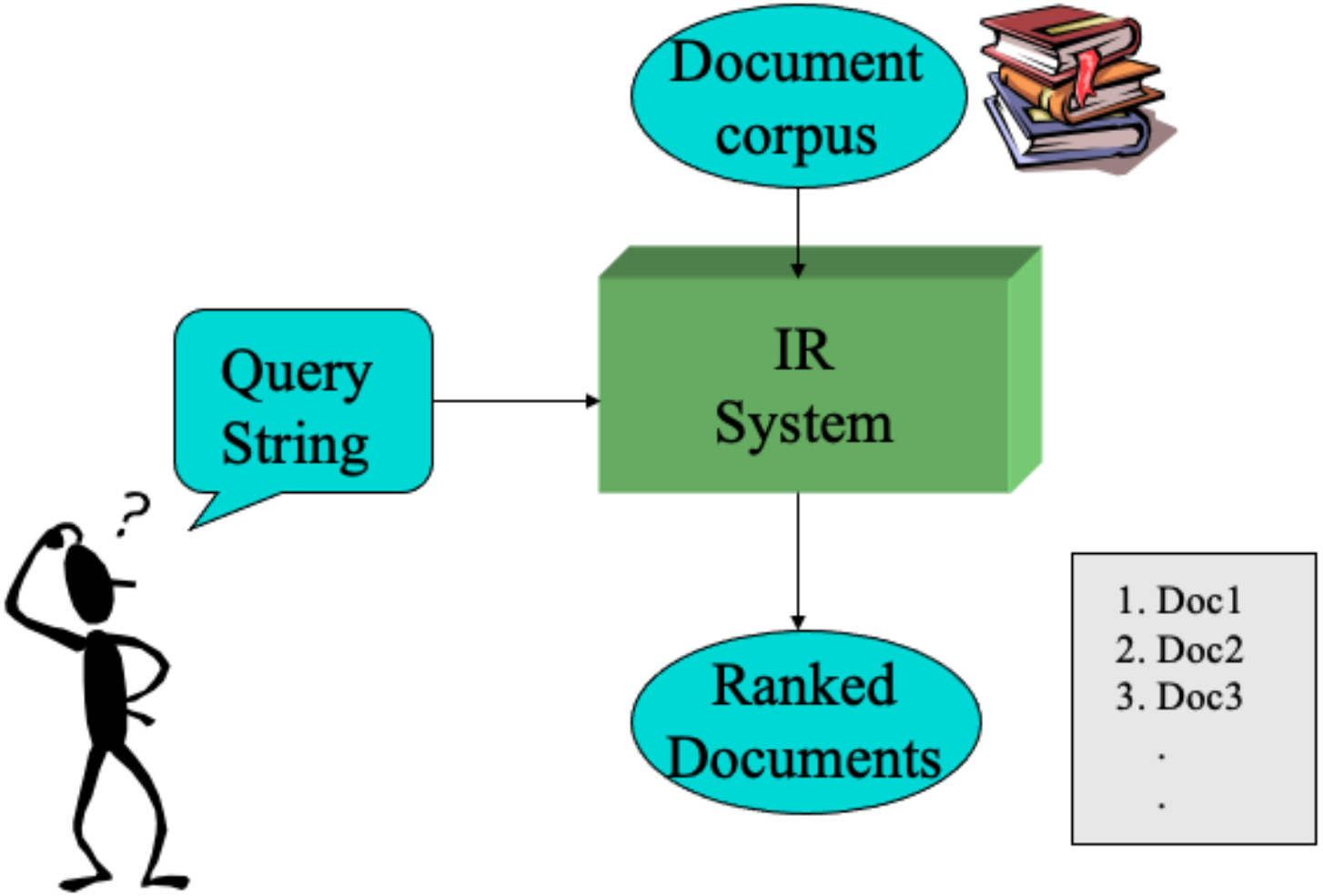
cataldo.musto@uniba.it

Credits: Marco Polignano

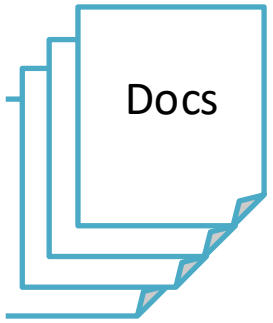
Recap

SEARCH ENGINE

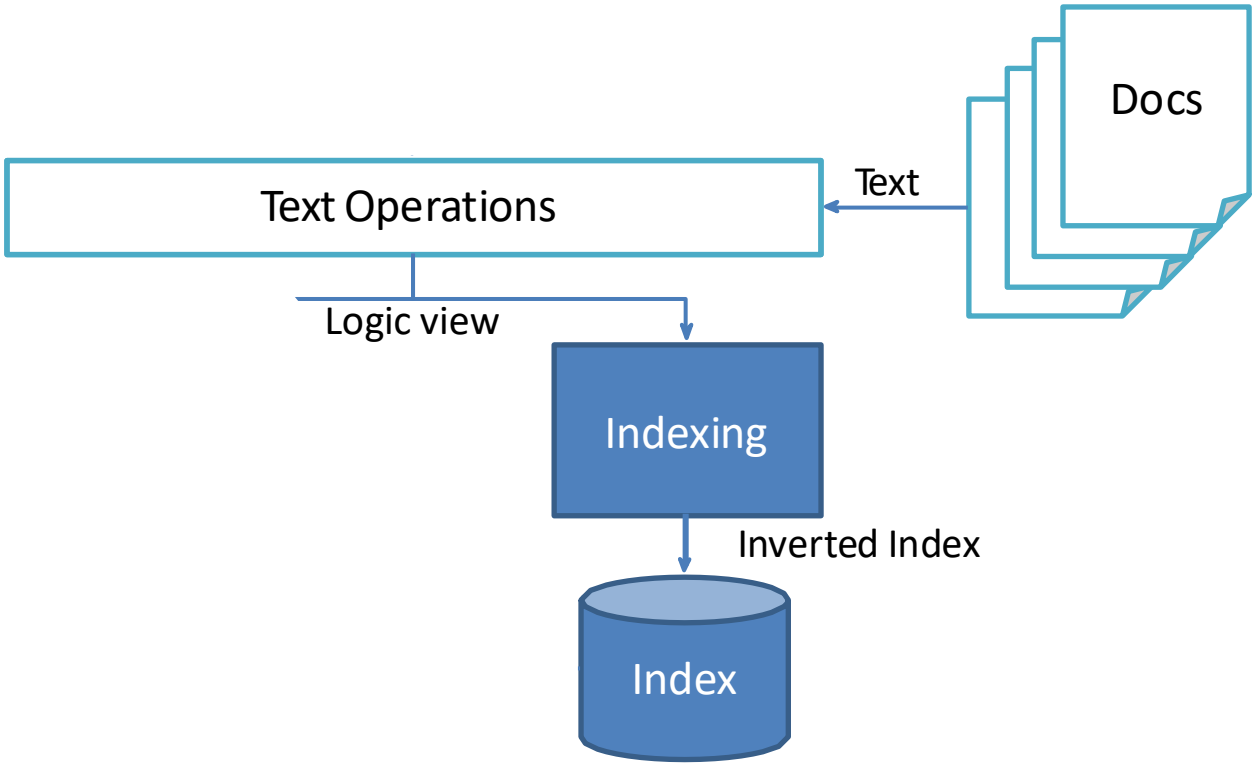
Information Retrieval Process (recap)



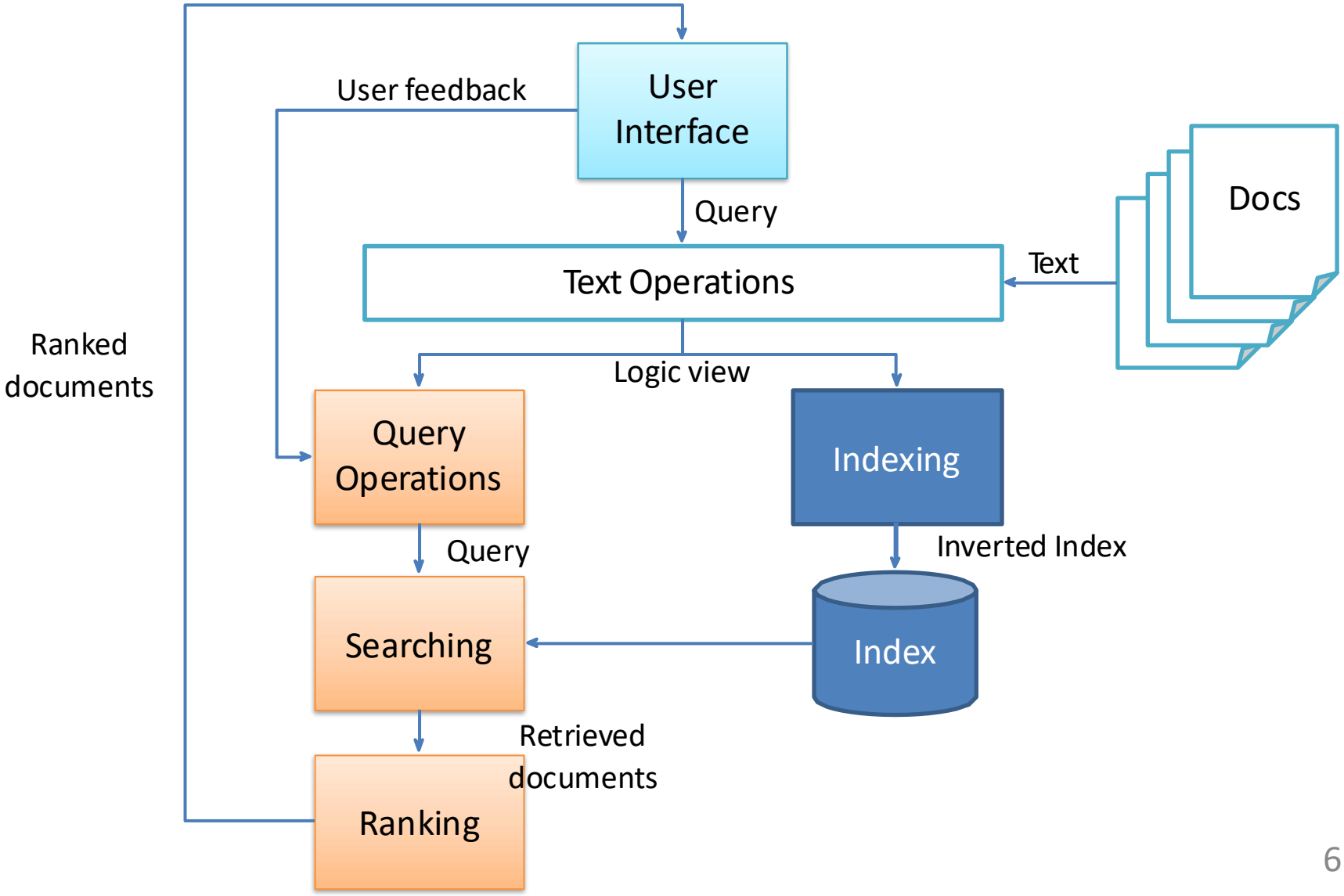
IR Process (at a lower level)



IR Process (at a lower level)



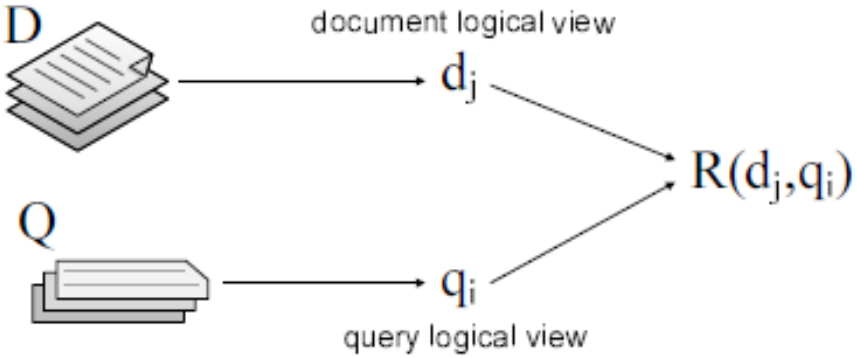
IR Process (at a lower level)



Information Retrieval Model

<D, Q, F, $R(q_i, d_j)$>

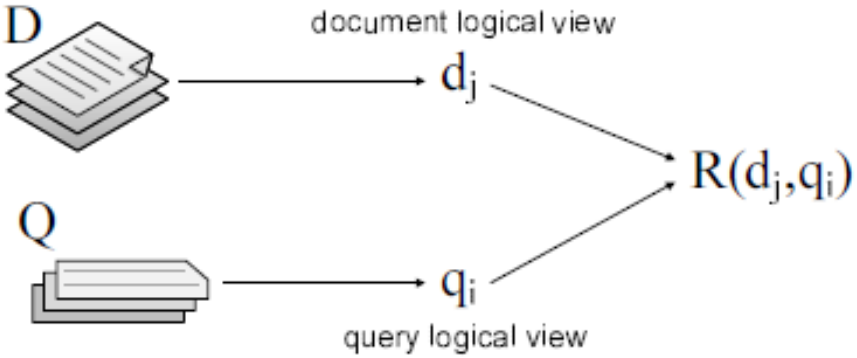
- D: document
- Q: query
- F: query/document representation function
- $R(q_i, d_j)$: ranking function



Information Retrieval Model

<D, Q, F, $R(q_i, d_j)$>

- D: document
- Q: query
- F: query/document representation function ?
- $R(q_i, d_j)$: ranking function



Representation

- **Representation is the way we transform documents and user queries into a form that a computer can process and compare.**

- The goal is to capture the **relevant meaning or content** of the original information while making it suitable for computational operations such as measuring similarity or ranking

- **Intuitively:**
  Representation is the bridge between human information and machine computation.

Bag-of-words representation

Document/query as unordered collection of words

John likes to watch movies. Mary likes too. John also likes to watch football games.

Bag-of-words representation

Document/query as unordered collection of words

John likes to watch movies. Mary likes too. John also likes to watch football games.



{"John": 1, "likes": 2, "to": 3, "watch": 4, "movies": 5, "also": 6, "football": 7, "games": 8, "Mary": 9, "too": 10}

Bag-of-words representation

Document/query as unordered collection of words

John likes to watch movies. Mary likes too. John also likes to watch football games.

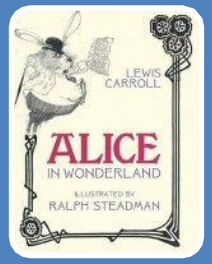


{"John": 1, "likes": 2, "to": 3, "watch": 4, "movies": 5, "also": 6, "football": 7, "games": 8, "Mary": 9, "too": 10}

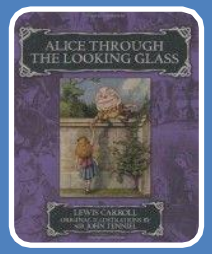
Token / Word

Position

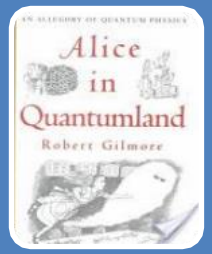
Term-Document matrix



- ‘It’s a friend of mine — a Cheshire Cat,’ said Alice: ‘allow me to introduce it.’
- ‘It’s the oldest rule in the book,’ said the King. ‘Then it ought to be Number One,’ said Alice.

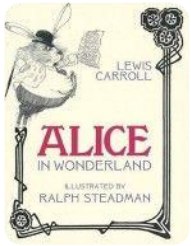


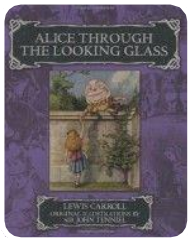
- Alice watched the White King as he slowly struggled up from bar to bar, till at last she said, ‘Why, you’ll be hours and hours getting to the table, at that rate.
- Alice looked round eagerly, and found that it was the Red Queen. ‘She’s grown a good deal!’ was her first remark.

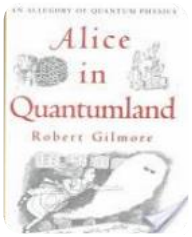


- In the pool of light was a billiards table, with two figures moving around it. Alice walked toward them, and as she approached they turned to look at her.
- Alice lay back, and closed her eyes. There was the Red Queen again, with that incessant grin. Or was it the Cheshire cat's grin?

Term-Document matrix







	D1	D2	D3
Cheshire Cat	1	0	1
Alice	2	2	2
book	1	0	0
King	1	1	0
table	0	1	1
Queen	0	1	1
grin	0	0	2

Term-Document matrix

	D1	D2	D3
Cheshire Cat	1	0	1
Alice	2	2	2
book	1	0	0
King	1	1	0
table	0	1	1
Queen	0	1	1
grin	0	0	2

Query: **Alice AND Queen**

Term-Document matrix



	D1	D2	D3	Q
Cheshire Cat	1	0	1	0
Alice	2	2	2	1
book	1	0	0	0
King	1	1	0	0
table	0	1	1	0
Queen	0	1	1	1
grin	0	0	2	0

Query: **Alice AND Queen**

Term-Document matrix



	D1	D2	D3	Q
Cheshire Cat	1	0	1	0
Alice	**2**	**2**	**2**	**1**
book	1	0	0	0
King	1	1	0	0
table	0	1	1	0
Queen	**0**	**1**	**1**	**1**
grin	0	0	2	0

Query: **Alice AND Queen**





This is a common representation that we adopt **for information retrieval tasks**

Term-Document matrix



	D1	D2	D3	Q
Cheshire Cat	1	0	1	0
Alice	**2**	**2**	**2**	**1**
book	1	0	0	0
King	1	1	0	0
table	0	1	1	0
Queen	**0**	**1**	**1**	**1**
grin	0	0	2	0

Query: **Alice AND Queen**





Result: D2, D3

This is a common representation that we adopt **for information retrieval tasks**

Inverted Index

Index

Page numbers in **bold face** refer to key term definitions
Page numbers in *italics* refer to images or diagrams
Page numbers followed by a "t" indicate a table

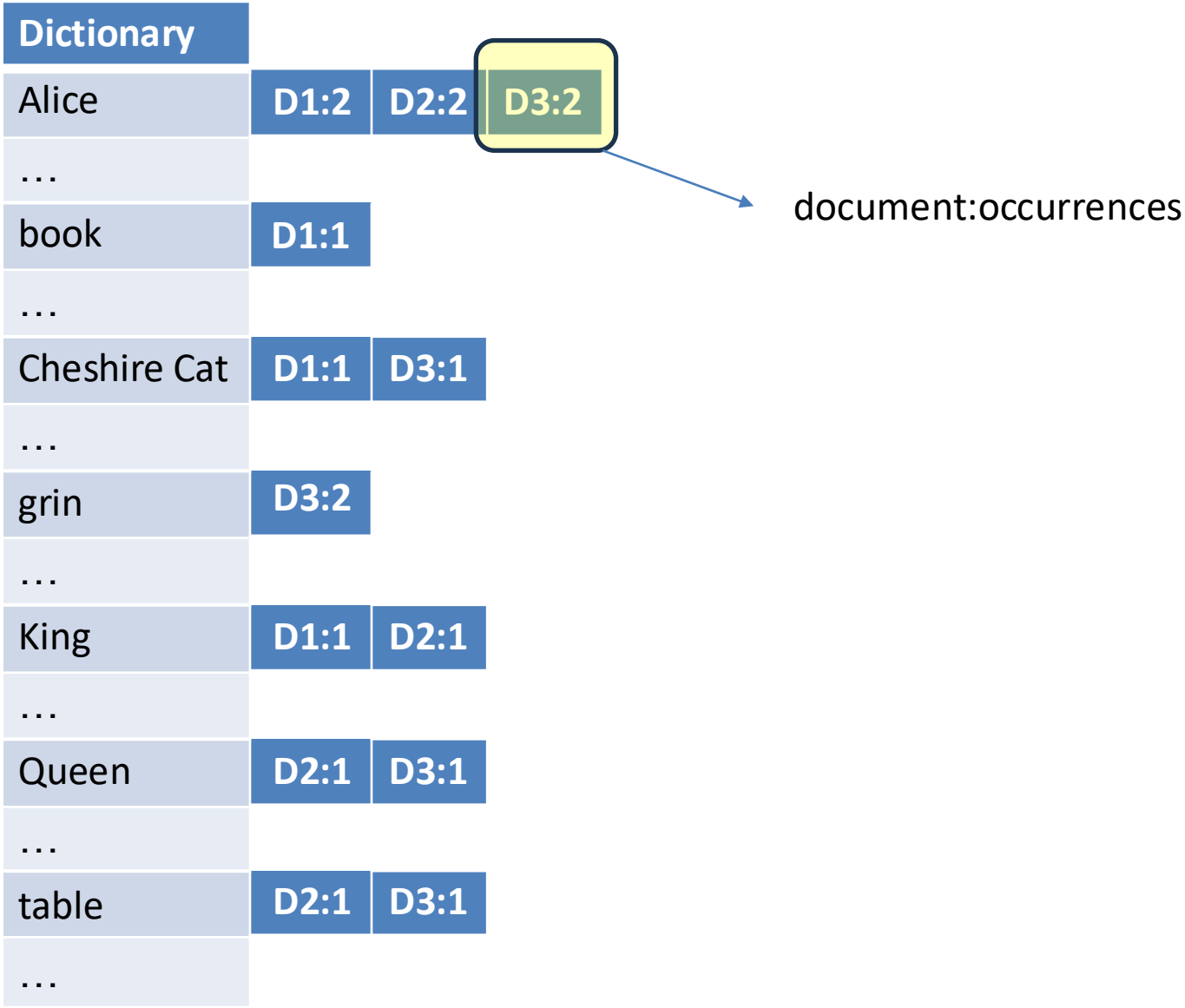
A

absolute temperature scale, **350**–351
absolute zero, **351**
acceleration of gravity, A.23t
accuracy, A.5
acetic acid (CH_3COOH)
  buffers, 575–576, 581–582
  conjugate acid-base pairs, 540
  ionization constant, 553, 554t
  manufacture of, 451
  titrations, 590–592
  as weak acid, 144t, 145, 551–552
acid-base pairs, conjugate, **540**–544
acid-base reactions, **538**
  autoionization of water, 545–547
  gas-forming exchange, 150–151
  net ionic equations for, 148–150
  neutralization, 146–150, 561–566t
  of salts, 146–151, 561–566
air, 342–343, 366–370, 380–381, 706
alkyl groups, **70**–71
alcohols, 64, 505–507
aldehydes, 278–279
alkali metals, **55**, 106
alkaline batteries, 670
alkaline earth metals, **55**
alkaline fuel cells, 674
alkalosis, 576
alkanes, **68**–71, 277–278, A.25–A.26
alkenes, **280**–283, A.26–A.27
alkyl group, **70**–71, A.25–A.26
alkynes, **281**, A.27
allotropes, **23**–24, 208, 403–*405*
alpha particles, 38–39, **693**–696, *697*, 699–700
alpha radiation, **693**
alpha rays, 36–37
aluminum (Al), 7, 8t, 103, 634–635, 682
amines, 544–**545**
ammonia (NH_3)
  amines, 545
  Brønsted-Lowry base, 538–539
  complex ions, 567
  ionization constant, 554t, 561

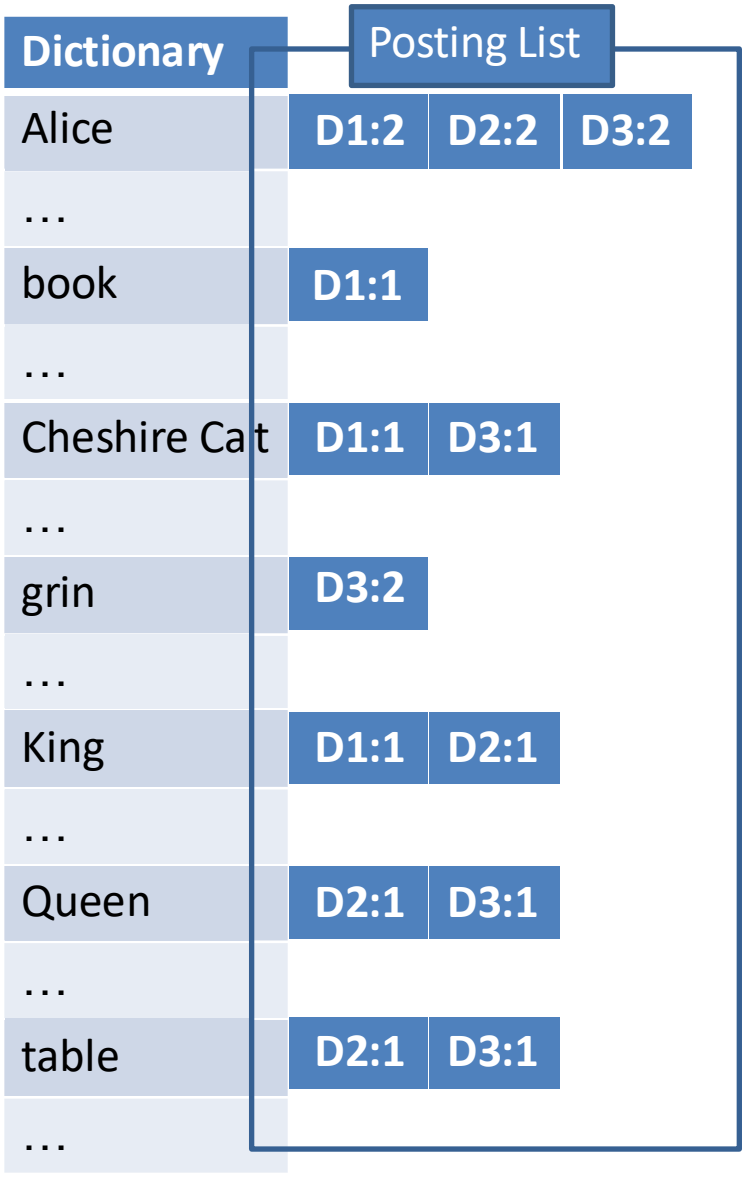
Inverted Index

Dictionary	
Alice	D1:2 D2:2 D3:2
…	
book	D1:1
…	
Cheshire Cat	D1:1 D3:1
…	
grin	D3:2
…	
King	D1:1 D2:1
…	
Queen	D2:1 D3:1
…	
table	D2:1 D3:1
…	

Inverted Index

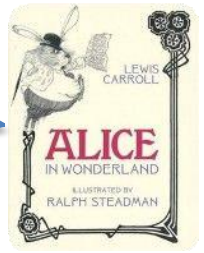


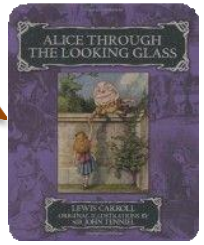
Inverted Index

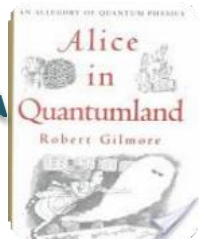


Inverted Index

Dictionary			
Alice	D1:2	D2:2	D3:2
…			
book	D1:1		
…			
Cheshire Cat	D1:1	D3:1	
…			
grin	D3:2		
…			
King	D1:1	D2:1	
…			
Queen	D2:1	D3:1	
…			
table	D2:1	D3:1	
…			

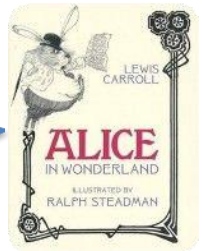


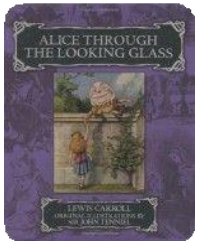


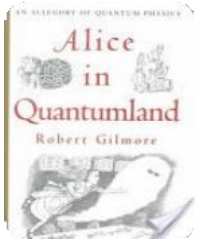


Inverted Index

Dictionary			
Alice	D1:2	D2:2	D3:2
…			
book	D1:1		
…			
Cheshire Cat	D1:1	D3:1	
…			
grin	D3:2		
…			
King	D1:1	D2:1	
…			
Queen	D2:1	D3:1	
…			
table	D2:1	D3:1	
…			

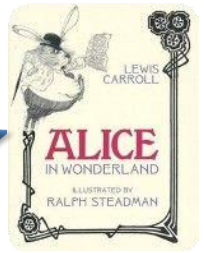


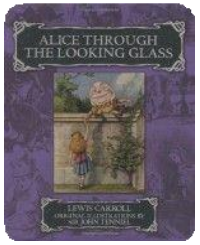


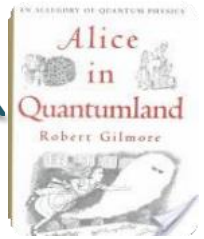


Inverted Index

Dictionary			
Alice	D1:2	D2:2	D3:2
…			
book	D1:1		
…			
Cheshire Cat	D1:1	D3:1	
…			
grin	D3:2		
…			
King	D1:1	D2:1	
…			
Queen	D2:1	D3:1	
…			
table	D2:1	D3:1	
…			

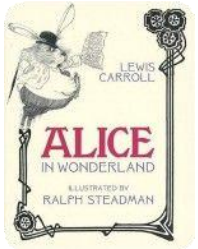


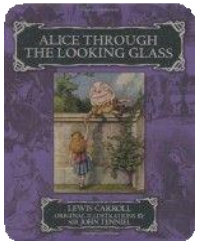


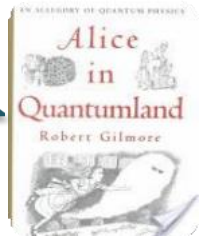


Inverted Index

Dictionary			
Alice	D1:2	D2:2	D3:2
…			
book	D1:1		
…			
Cheshire Cat	D1:1	D3:1	
…			
grin	D3:2		
…			
King	D1:1	D2:1	
…			
Queen	D2:1	D3:1	
…			
table	D2:1	D3:1	
…			

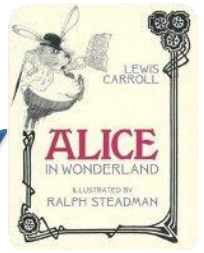


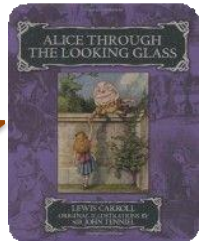


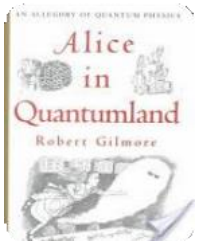


Inverted Index

Dictionary			
Alice	D1:2	D2:2	D3:2
…			
book	D1:1		
…			
Cheshire Cat	D1:1	D3:1	
…			
grin	D3:2		
…			
King	D1:1	D2:1	
…			
Queen	D2:1	D3:1	
…			
table	D2:1	D3:1	
…			

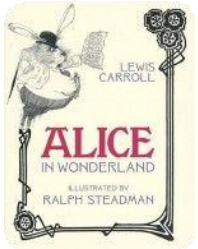


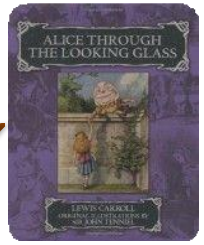


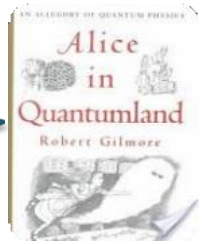


Inverted Index

Dictionary	
Alice	D1:2 D2:2 D3:2
…	
book	D1:1
…	
Cheshire Cat	D1:1 D3:1
…	
grin	D3:2
…	
King	D1:1 D2:1
…	
Queen	D2:1 D3:1
…	
table	D2:1 D3:1
…	

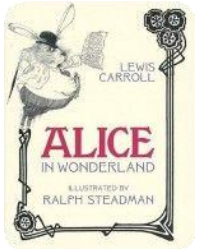


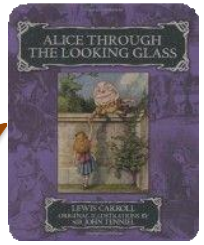


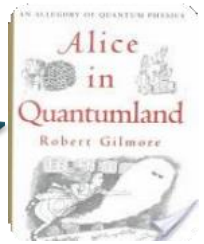


Inverted Index

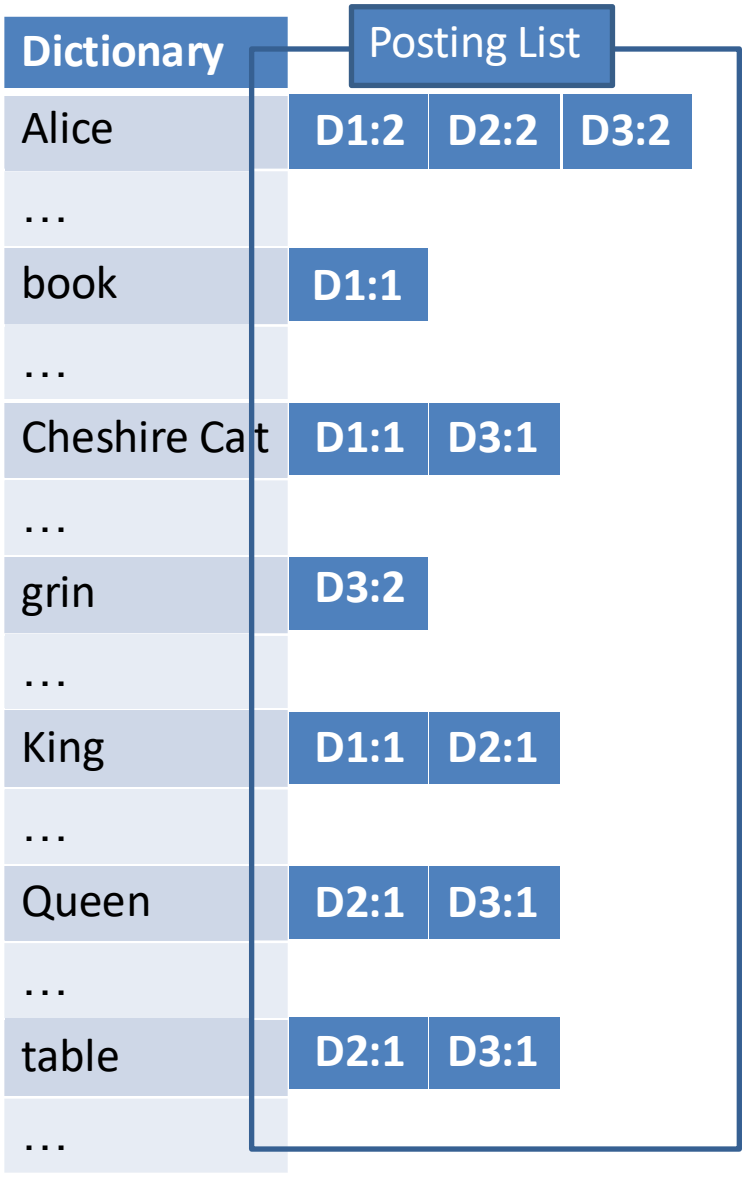
Dictionary			
Alice	D1:2	D2:2	D3:2
…			
book	D1:1		
…			
Cheshire Cat	D1:1	D3:1	
…			
grin	D3:2		
…			
King	D1:1	D2:1	
…			
Queen	D2:1	D3:1	
…			
table	D2:1	D3:1	
…			





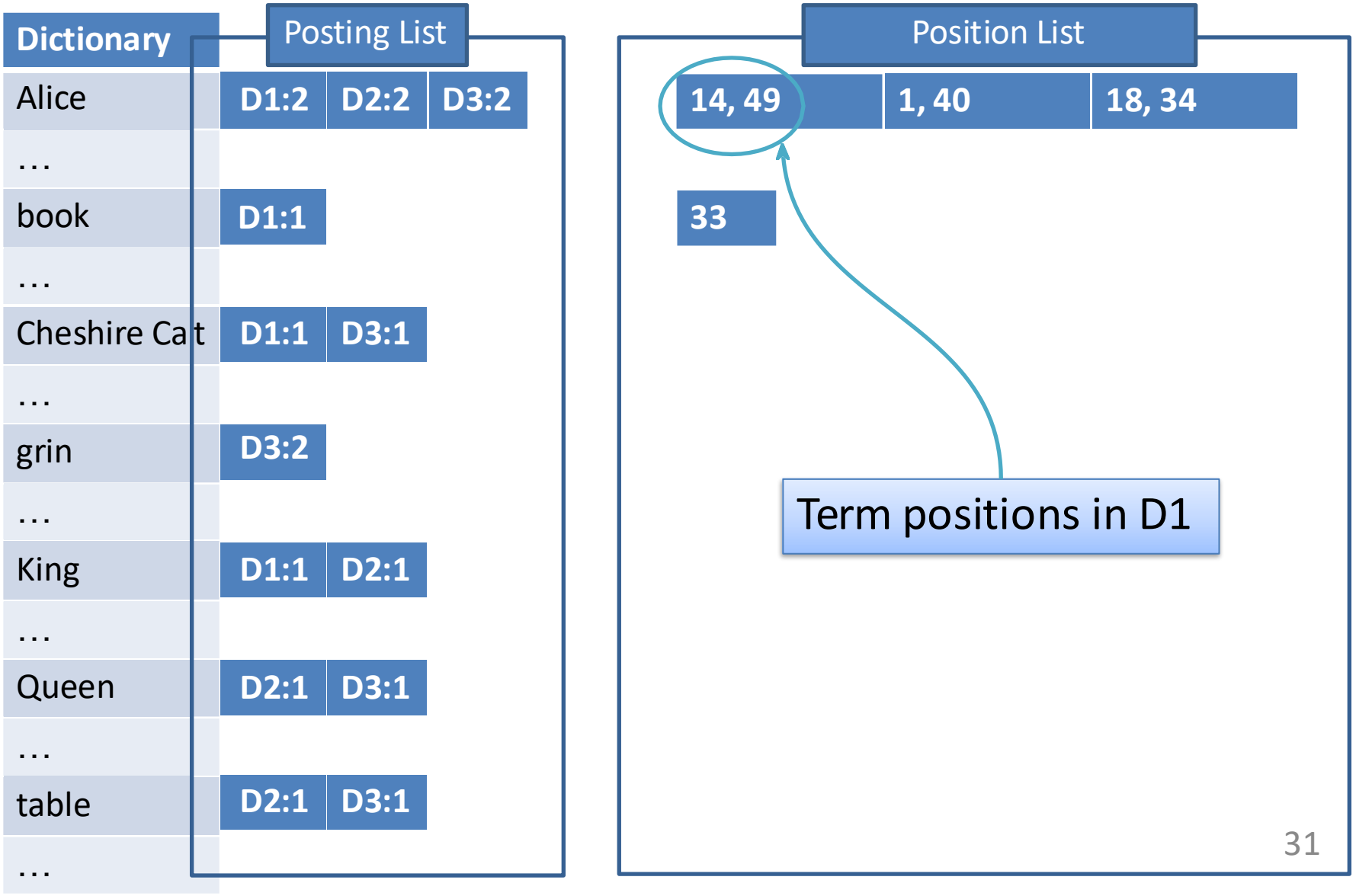


Inverted Index

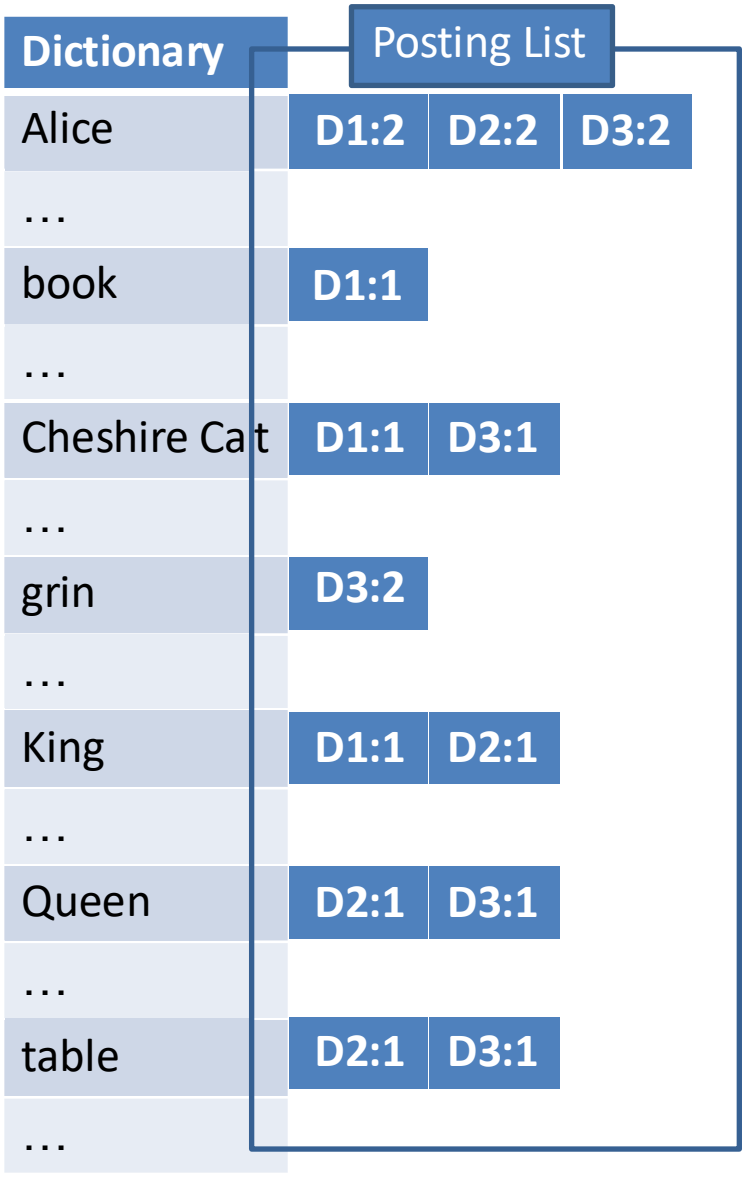


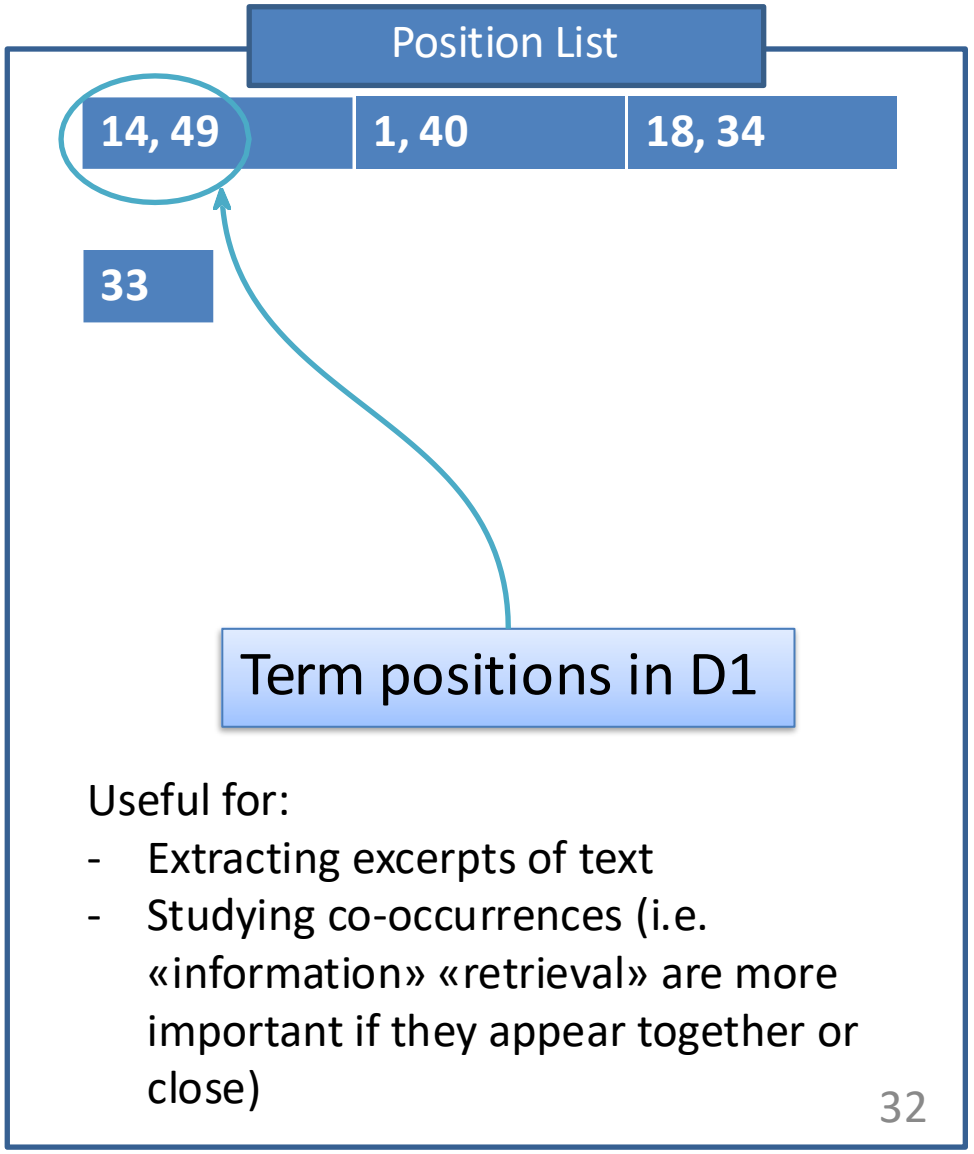
In more advanced models, we also take into account **the precise position** of the token in the document.

Inverted Index



Inverted Index





Useful for:

- Extracting excerpts of text
- Studying co-occurrences (i.e. «information» «retrieval» are more important if they appear together or close)

Inverted Index: why?

- For computational reasons

- Without inverted index a search algorithm shall go **through all the documents** in the corpus to check whether the query term occurs or not
  - Big corpora (i.e., web search) → huge computational costs!

Inverted Index: why?



	D1	D2	D3	Q
Cheshire Cat	1	0	1	0
Alice	**2**	**2**	**2**	**1**
book	1	0	0	0
King	1	1	0	0
table	0	1	1	0
Queen	**0**	**1**	**1**	**1**
grin	0	0	2	0

Query: **Alice AND Queen**





Result: D2, D3

This is a common representation that we adopt **for information retrieval tasks**

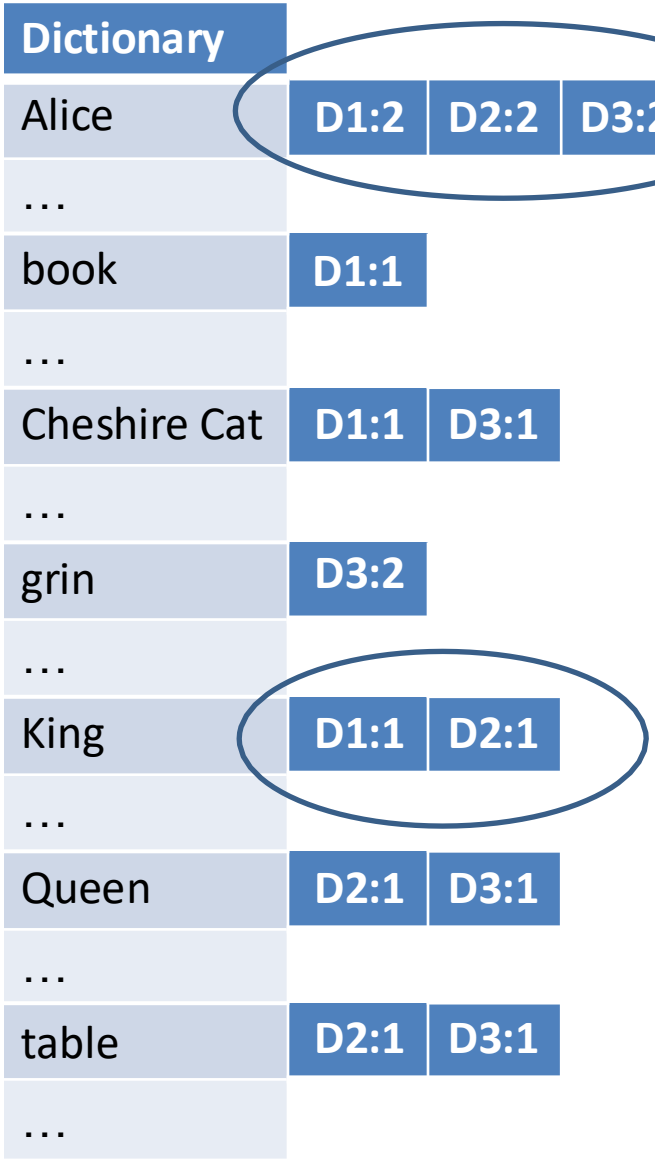
Inverted Index: query processing

Dictionary	
Alice	D1:2 D2:2 D3:2
…	
book	D1:1
…	
Cheshire Cat	D1:1 D3:1
…	
grin	D3:2
…	
King	D1:1 D2:1
…	
Queen	D2:1 D3:1
…	
table	D2:1 D3:1
…	

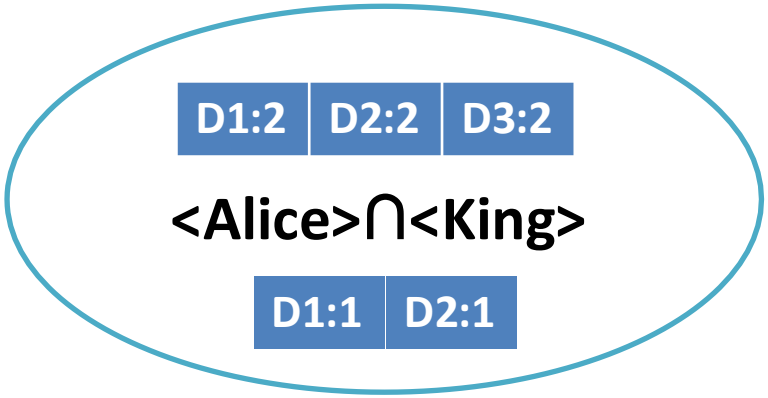
AND (intersection ∩)

<Alice>∩<King>

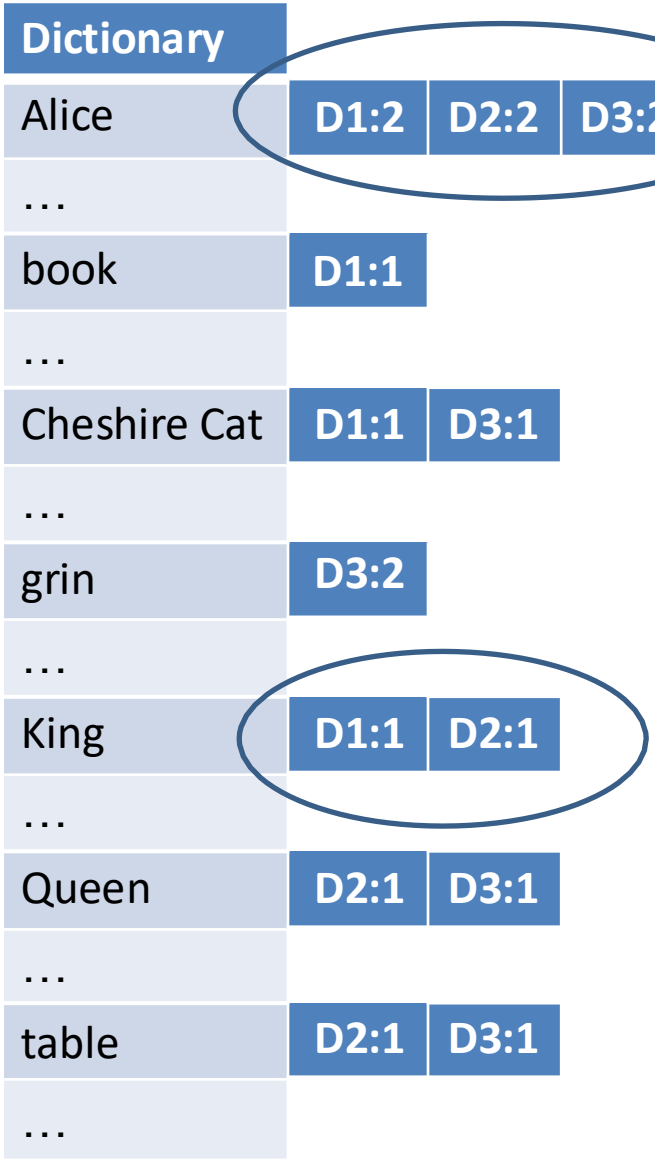
Inverted Index: query processing



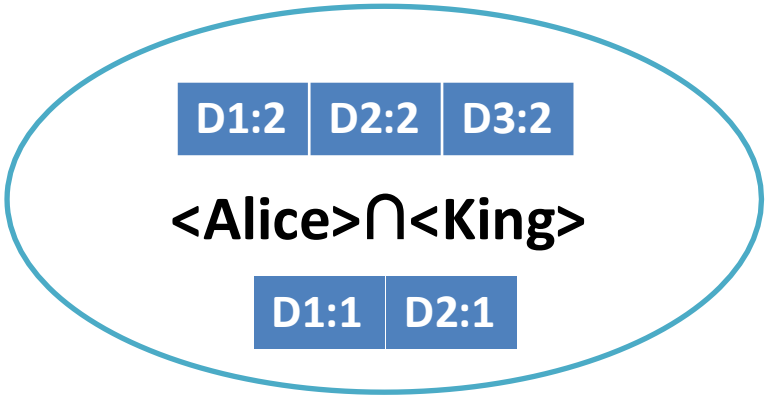
AND (intersection ∩)



Inverted Index: query processing



AND (intersection ∩)



Alice AND King -> (D1, D2)

Inverted Index: query processing

Dictionary	
Alice	D1:2 D2:2 D3:2
…	
book	D1:1
…	
Cheshire Cat	D1:1 D3:1
…	
grin	D3:2
…	
King	D1:1 D2:1
…	
Queen	D2:1 D3:1
…	
table	D2:1 D3:1
…	

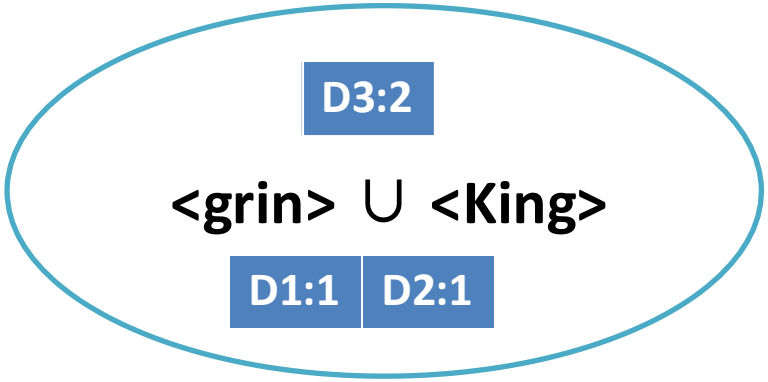
OR (union ∪)

<grin> ∪ <King>

Inverted Index: query processing

Dictionary	
Alice	D1:2 D2:2 D3:2
…	
book	D1:1
…	
Cheshire Cat	D1:1 D3:1
…	
grin	D3:2
…	
King	D1:1 D2:1
…	
Queen	D2:1 D3:1
…	
table	D2:1 D3:1
…	

OR (union ∪)



grin OR King -> (D1, D2, D3)

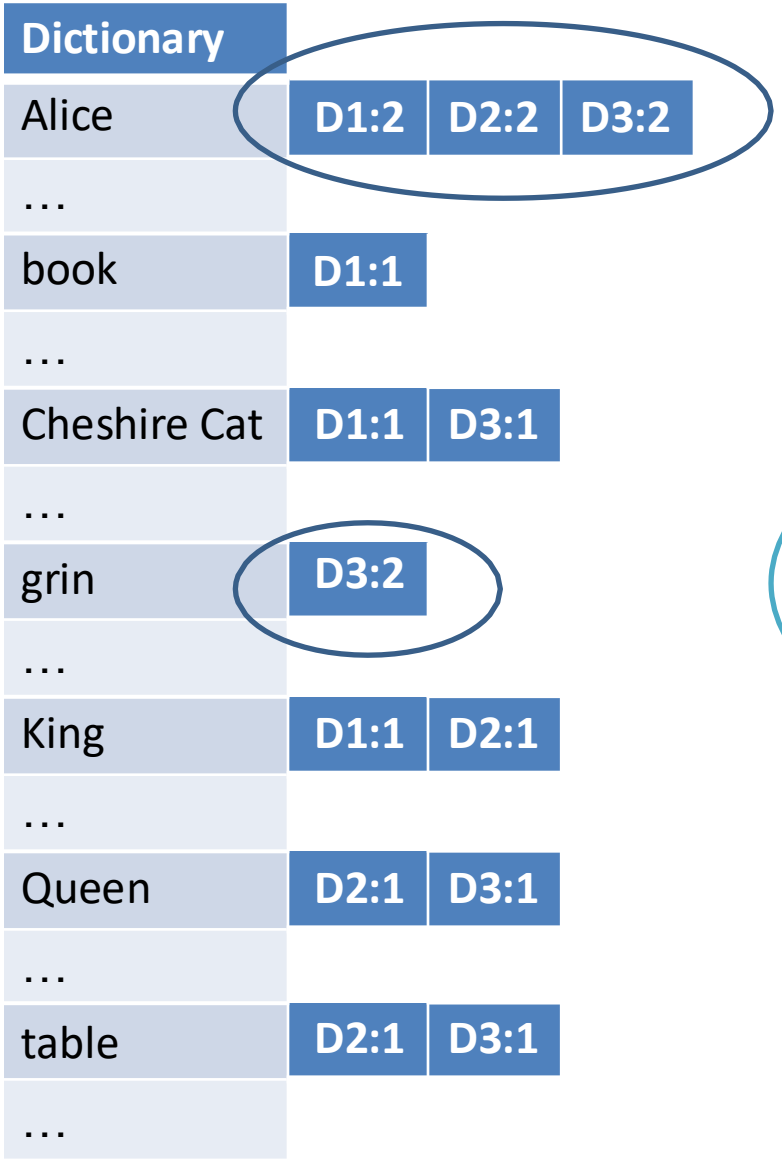
Inverted Index: query processing

Dictionary	
Alice	D1:2 D2:2 D3:2
…	
book	D1:1
…	
Cheshire Cat	D1:1 D3:1
…	
grin	D3:2
…	
King	D1:1 D2:1
…	
Queen	D2:1 D3:1
…	
table	D2:1 D3:1
…	

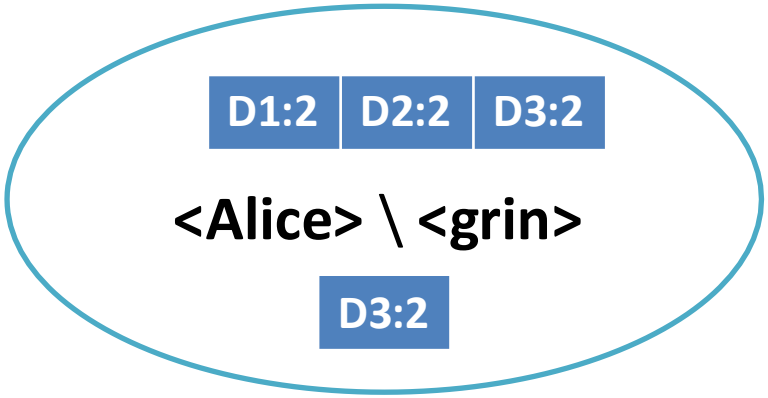
NOT (complement \)

<Alice> \ <grin>

Inverted Index: query processing



NOT (complement \)



Alice NOT grin -> (D1, D2)

Term-weight

- Thanks to the inverted index, we know the **«candidate» documents**.
  - Next: we need to rank the documents
- Intuition
  - We can count term occurrences
    - The more the occurrences, the more the relevance
    - This can be done through TF
      - TF (term frequency): term occurrences in the document

Term-weight

- Measures the term relevance in a document
  - component value in the document representation
  - TF
    - TF (term frequency): term occurrences in the document

	D1	D2	D3	Q
Cheshire Cat	1	0	1	0
Alice	2	2	2	1
book	1	0	0	0
King	1	1	0	0
table	0	1	1	0
Queen	0	1	1	1
grin	0	0	2	0

Term-weight

- Thanks to the inverted index, we know the «candidate» documents.
  - Next: we need to rank the documents
- Term-weight Measures the term relevance in a document
  - TF
    - TF (term frequency): term occurrences in the document

- **Is there any problem with counting?**

Term-weight

- Measures the term relevance in a document
  - component value in the document representation
  - TF
    - TF (term frequency): term occurrences in the document
- Is there any problem with simple counting?
  - Yes, poorly informative words have high frequency

A toy example

D1
In the morning the girl walked through the garden. The air in the garden was fresh and the sun was bright.

D2
The garden was large and full of colorful flowers. In the garden she found a small key on the ground.

D3
The white rabbit ran across the field and disappeared into the forest.

D4
Alice met the queen in the castle. The queen gave her a card and the game began.

D5
A small bird was singing in the tree near the garden where the rabbit had been.

A toy example

D1
In the morning the girl walked through the garden. The air in the garden was fresh and the sun was bright.

D2
The garden was large and full of colorful flowers. In the garden she found a small key on the ground.

D3
The white rabbit ran across the field and disappeared into the forest.

D4
Alice met the queen in the castle. The queen gave her a card and the game began.

D5
A small bird was singing in the tree near the garden where the rabbit had been.

Term	D1 (tf)	D2 (tf)	D3 (tf)	D4 (tf)	D5 (tf)
the	3	1	2	3	2
garden	2	3	0	0	1
rabbit	0	0	1	0	1
alice	0	0	0	1	0
queen	0	0	0	1	0
tree	0	0	0	0	1

A toy example

D1
In the morning the girl walked through the garden. The air in the garden was fresh and the sun was bright.

D2
The garden was large and full of colorful flowers. In the garden she found a small key on the ground.

D3
The white rabbit ran across the field and disappeared into the forest.

D4
Alice met the queen in the castle. The queen gave her a card and the game began.

D5
A small bird was singing in the tree near the garden where the rabbit had been.

Term	D1 (tf)	D2 (tf)	D3 (tf)	D4 (tf)	D5 (tf)
the	3	1	2	3	2
garden	2	3	0	0	1
rabbit	0	0	1	0	1
alice	0	0	0	1	0
queen	0	0	0	1	0
tree	0	0	0	0	1

Query

"the garden"

Ranking using only term frequency (TF)

Score = tf(the) + tf(garden)

Rank	Document	tf(the)	tf(garden)	Score
1	D1	3	2	5
2	D2	1	3	4
3	D4	3	0	3
4	D5	2	1	3
5	D3	2	0	2

Term-weight

- Measures the term relevance in a document
  - component value in the document representation
  - TF*IDF
    - TF (term frequency): term occurrences in the document
    - IDF (inverse document frequency): inverse to the number of documents in which the term occurs

$$tf * idf(t,d) = tf(t,d) * \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

Term-weight

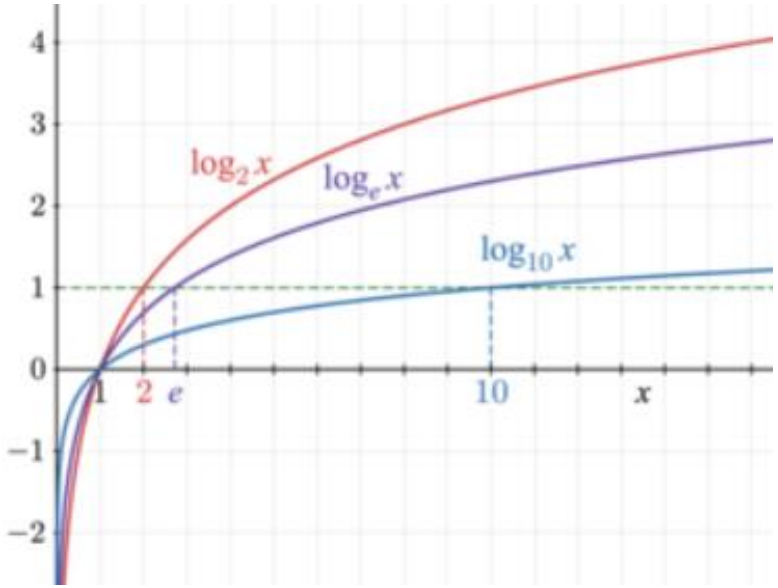
- Measures the term relevance in a document
  - component value in the document representation
  - TF*IDF
    - TF (term frequency): term occurrences in the document
    - IDF (inverse document frequency): inverse to the number of documents in which the term occurs

$$tf * idf(t,d) = tf(t,d) * \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

Number of documents in the collection

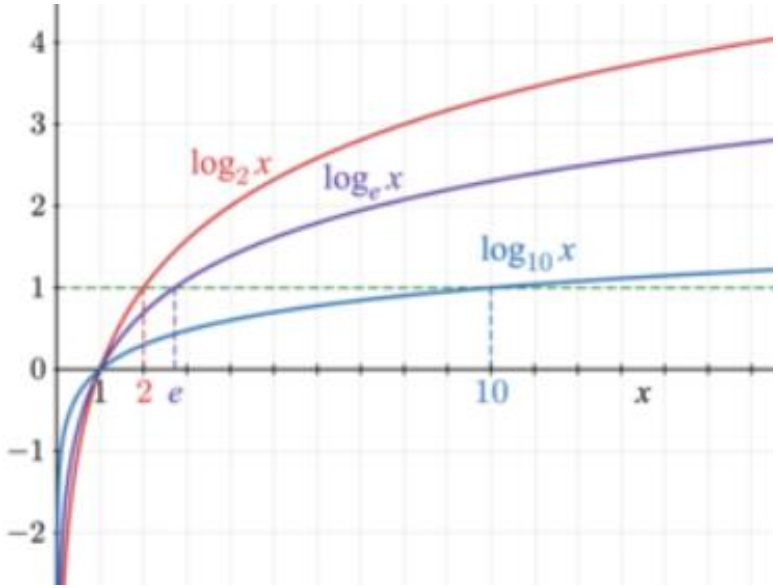
idf

Term-weight



$$tf * idf(t,d) = tf(t,d) * \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

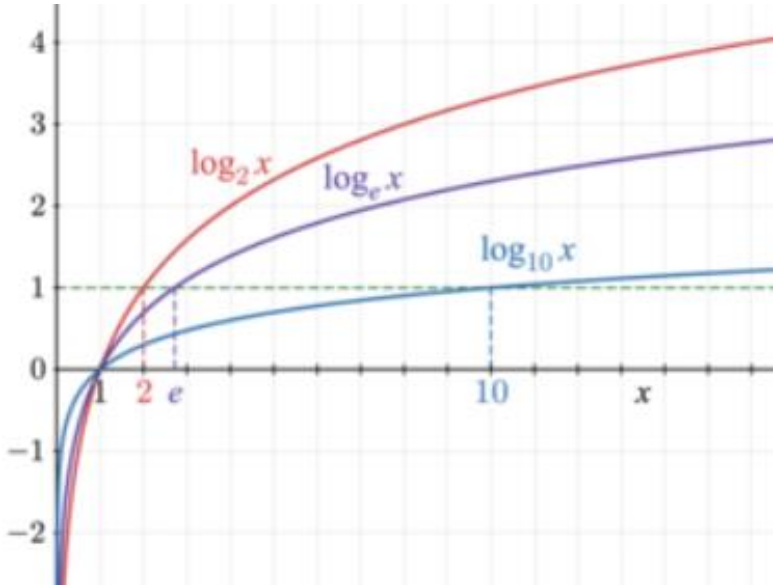
Term-weight



What happens if a term appears in many documents?

$$tf * idf(t,d) = tf(t,d) * \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

Term-weight

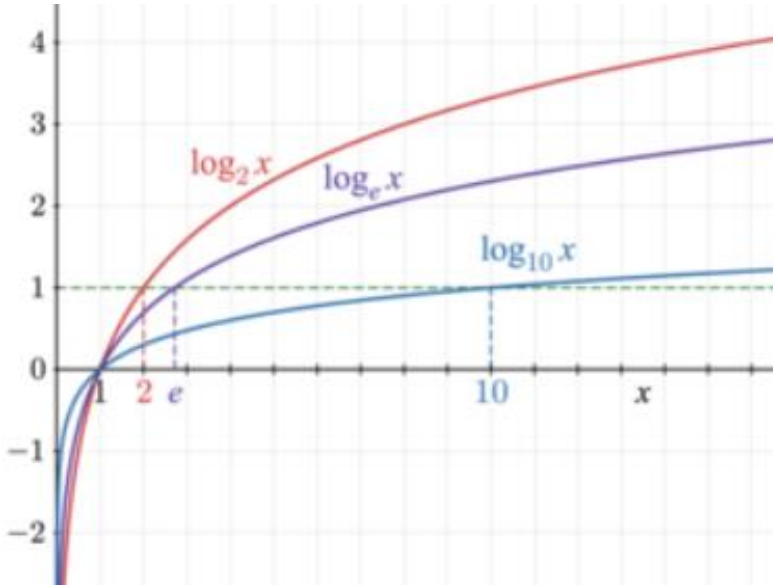


What happens if a term appears in many documents?

The ratio is close to 1, so the logarithm is close to 0

$$tf * idf(t,d) = tf(t,d) * \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

Term-weight



What happens if a term appears in many documents?

The ratio is close to 1, so the logarithm is close to 0

Conversely, if a term appears in just a few document, **the ratio is high and the logarithm is high as well**

$$tf * idf(t,d) = tf(t,d) * \log \frac{|D|}{|\{d \in D : t \in d\}|}$$

TF*IDF insights

- **increases proportionally** to the term frequency in the document
- **decreases** to the number of documents in which the term belongs
  - common words are generally more frequent in the collection
- **IDF** depends on the collection, **TF** on the document

Inverted index/TF*IDF

- TF: computed by term occurrences in the posting list
- IDF: computed by the posting list cardinality

Alice	D1:2	D2:2	D3:2

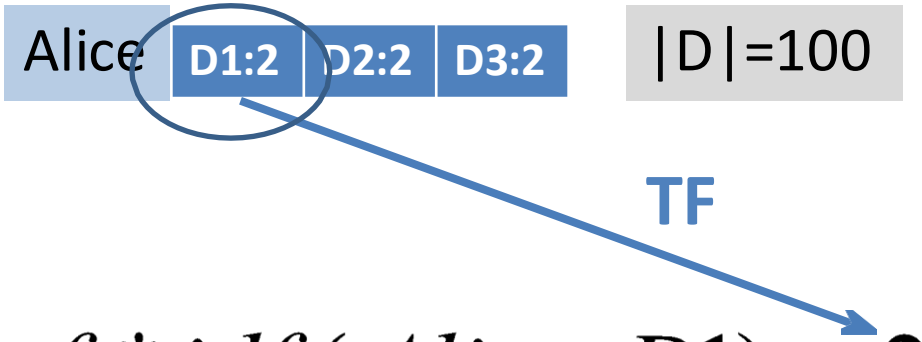
|D|=100

$$\log \frac{|D|}{|\{d \in D : t \in d\}|}$$

$$tf * idf(Alice, D1) =$$

Inverted index/TF*IDF

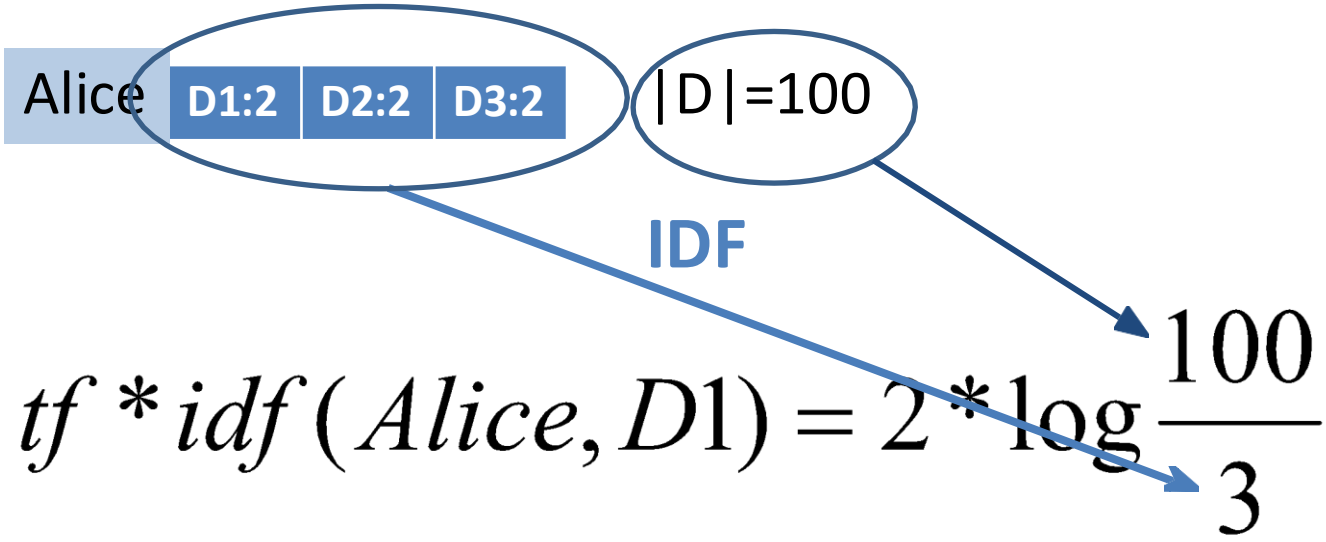
- TF: computed by term occurrences in the posting list
- IDF: computed by the posting list cardinality



$$tf * idf(Alice, D1) = 2 * \log \frac{100}{3}$$

Inverted index/TF*IDF

- TF: computed by term occurrences in the posting list
- IDF: computed by the posting list cardinality



$$tf * idf(Alice, D1) = 2 * \log \frac{100}{3}$$

Inverted index/TF*IDF

D1
In the morning the girl walked through the garden. The air in the garden was fresh and the sun was bright.
(the: 3, garden: 2)

D2
The garden was large and full of colorful flowers. In the garden she found a small key on the ground.
(the: 1, garden: 3)

D3
The white rabbit ran across the field and disappeared into the forest.
(the: 2, garden: 0)

D4
Alice met the queen in the castle. The queen gave her a card and the game began.
(the: 3, garden: 0)

D5
A small bird was singing in the tree near the garden where the rabbit had been.
(the: 2, garden: 1)

Term	D1 (tf)	D2 (tf)	D3 (tf)	D4 (tf)	D5 (tf)	Document frequency (df)	IDF (log N/df)
the	3	1	2	3	2	5	0.00
garden	2	3	0	0	1	3	0.22
rabbit	0	0	1	0	1	2	0.40
alice	0	0	0	1	0	1	0.70
queen	0	0	0	1	0	1	0.70
tree	0	0	0	0	1	1	0.70

Query
"the garden"

Ranking using only term frequency (TF)
Score = tf(the) + tf(garden)

Rank	Document	tf(the)	tf(garden)	Score
1	D1	3	2	5
2	D2	1	3	4
3	D4	3	0	3
4	D5	2	1	3
5	D3	2	0	2

Ranking using TF–IDF
Score = tf(the) · IDF(the) + tf(garden) · IDF(garden)

Rank	Document	tf(the) · IDF	tf(garden) · IDF	Score
1	D2	1 · 0.00 = 0.00	3 · 0.22 = 0.66	0.66
2	D1	3 · 0.00 = 0.00	2 · 0.22 = 0.44	0.44
3	D5	2 · 0.00 = 0.00	1 · 0.22 = 0.22	0.22
4	D4	3 · 0.00 = 0.00	0 · 0.22 = 0.00	0.00
5	D3	2 · 0.00 = 0.00	0 · 0.22 = 0.00	0.00

Using IDF changes the ranking: with TF, D1 is ranked first because it contains more occurrences of the query terms, but with TF–IDF the common word "the" gets zero weight and D2 becomes the most relevant document because it contains more occurrences of "garden".